

学校名称（占位）

院系名称（占位）

面向小型超市的库存与缺货预警系统

英文论文逻辑对照稿

作者姓名（占位）

学号（占位）

指导教师（占位）

本稿用于中文阅读和论证检查，不替代英文提交版本。

2026 年 10 月

说明

本稿与英文论文使用同一套系统、代码、测试和实验数据，目的在于帮助读者用中文检查论文的逻辑链条。中文稿不重新提出一套不同的方法，也不增加英文论文中没有验证的结论。作者、学校、院系、学位、导师和学号等信息暂时保留占位符。

论文的主线可以概括为：小型超市需要的不只是一个“当前库存数”，还需要知道库存为什么变化、谁进行了操作，以及哪些商品需要处理。因此，系统把库存数量、库存流水、预警生命周期和人工确认分开建模，再通过数据库事务、行锁、幂等键和可失效缓存保证这几类信息不会互相矛盾。论文后半部分不把系统功能直接当成研究结论，而是使用测试和实验逐项验证前面的设计目标。

目录

1 问题、目标与研究问题	1
2 需求与验收标准	2
3 系统设计	3
4 系统实现	5
5 测试与实验	7
6 讨论、局限与结论	8

1 问题、目标与研究问题

1.1 问题背景

小型超市的库存变化频繁而且来源多样：早上的送货会增加库存，销售或出库会减少库存，人工盘点可能发现差异，损耗又可能需要单独调整。如果系统只保存一个可以直接编辑的数量，工作人员无法在事后区分“收到了一批货”“卖出了一件商品”“盘点修正了数量”或者“商品损坏”。数量即使暂时正确，解释也已经丢失。

缺货预警还有一个容易被忽略的逻辑问题。店长点击“已查看”并不会让商品重新上架；相反，补货成功也不一定意味着有人点击了确认按钮。如果把确认动作直接当成预警解决，系统会把人的处理行为和库存事实混在一起。因此，本项目把“库存是否恢复”和“谁查看过预警”建模为两个相互独立的维度。

1.2 研究目标

项目目标是实现并验证一个单店库存与缺货预警系统 Shelfwise。系统需要满足以下条件：库存变化必须能够追溯；并发或重复请求不能把库存扣成负数或重复记账；预警状态应该能解释库存状态和人工处理状态；Redis 只能作为查询加速手段，不能成为库存正确性的前提；其他开发者能够从干净检出开始复现主要检查和论文构建。

1.3 研究问题

表 1: 研究问题与证据对应关系

问题	关注点	主要证据
RQ1	重复提交和并发出库下，库存是否保持非负并保留可归因历史？	MySQL/Redis 集成测试、库存流水、并发场景
RQ2	缺货、补货和人工确认能否表示为可观察的不同事件？	预警状态迁移、阈值测试、预警界面和确认测试
RQ3	Redis 缓存对本地预警查询有什么影响，Redis 故障时结果是否保持？	交替缓存实验、响应等价性、Redis 停机恢复实验
RQ4	其他开发者能否复现系统检查和论文产物？	版本矩阵、迁移脚本、CI、构建和证据脚本

这些问题有意保持为软件工程问题。RQ1 不是证明所有硬件故障下的绝对正确性；RQ2 不是用户满意度调查；RQ3 不是生产环境 SLA；RQ4 也不是保证任何机器都能零配置运行。每个问题都限定了研究对象、实验环境和证据类型，从而避免让论文结论超过实现和实验实际支持的范围。

1.4 范围边界

系统面向一个小型超市，支持管理员、店长和库存员三个角色。系统包含商品、分类、供应商、库存事务、库存流水、缺货预警、预警确认、报表和基础设置。系统不包含多店租户、在线支付、财务核算、采购结算、自动采购订单和需求预测；库存使用整数单位，不处理小数商品和批次有效期。

库存价值仅按当前数量乘以当前商品单价计算，是一个展示估计，不是会计成本或历史估值。补货阈值由店长维护，系统验证阈值之间的数值关系，但不声称能够自动推导最优阈值。

2需求与验收标准

2.1 角色职责

管理员负责产品、分类、供应商、用户和设置；店长查看库存和报表、调整安全库存与补货阈值、查看并确认预警；库存员登记入库、出库、调整和盘点。三类用户都可以查询库存和库存流水，但服务器仍然逐请求检查权限，前端隐藏菜单不是唯一的安全边界。

表 2: 角色权限矩阵				
操作	管理员	店长	库存员	
查询库存和库存流水	允许	允许	允许	
登记入库、出库、调整、盘点	允许	不允许	允许	
维护商品、分类、供应商	允许	不允许	不允许	
调整安全库存和补货阈值	允许	允许	不允许	
查看和确认预警	允许	允许	不允许	
查看报表	允许	允许	不允许	
管理用户和系统设置	允许	不允许	不允许	

2.2 库存不变量

库存事务使用如下关系：

$$q_{new} = q_{old} + \Delta, \quad 0 \leq q_{new} \leq 1,000,000,000$$

入库和出库的输入数量为正数，由服务层决定符号；调整使用带符号数量；盘点使用盘点后的绝对数量，系统在持有产品行锁时计算差值。库存数量不能通过商品编辑接口直接覆盖。

每条成功流水至少包含产品、操作人、事务类型、变更前数量、变更量、变更后数量、原因、幂等键和服务器时间。原因必须存在，但原因文字本身不能证明现实中的送货或盘点一定发生，因此论文只把它称为可追溯记录，不称为不可篡改的法律证据。

2.3 预警语义

数量为 0 时生成 OUT；数量大于 0 且小于等于补货阈值时生成 LOW；数量高于补货阈值时处于健康状态。缺货恢复后关闭原缺货事件，并创建 24 小时有效的 RESTOCKED 事件。确认预警只增加确认人和确认时间，不改变数量，也不自动关闭预警。状态变化、阈值修改和归档操作都需要重新计算预警。

3 系统设计

3.1 整体架构

系统是单体模块化架构：Vue 3 浏览器端负责界面、路由、表单和状态展示；Spring Boot 负责 REST API、认证、权限、业务事务和校验；MySQL 保存持久化数据；Redis 缓存预警查询结果。MySQL 是最终事实来源，Redis 故障时查询可以回退到 MySQL。

Shelfwise runtime architecture

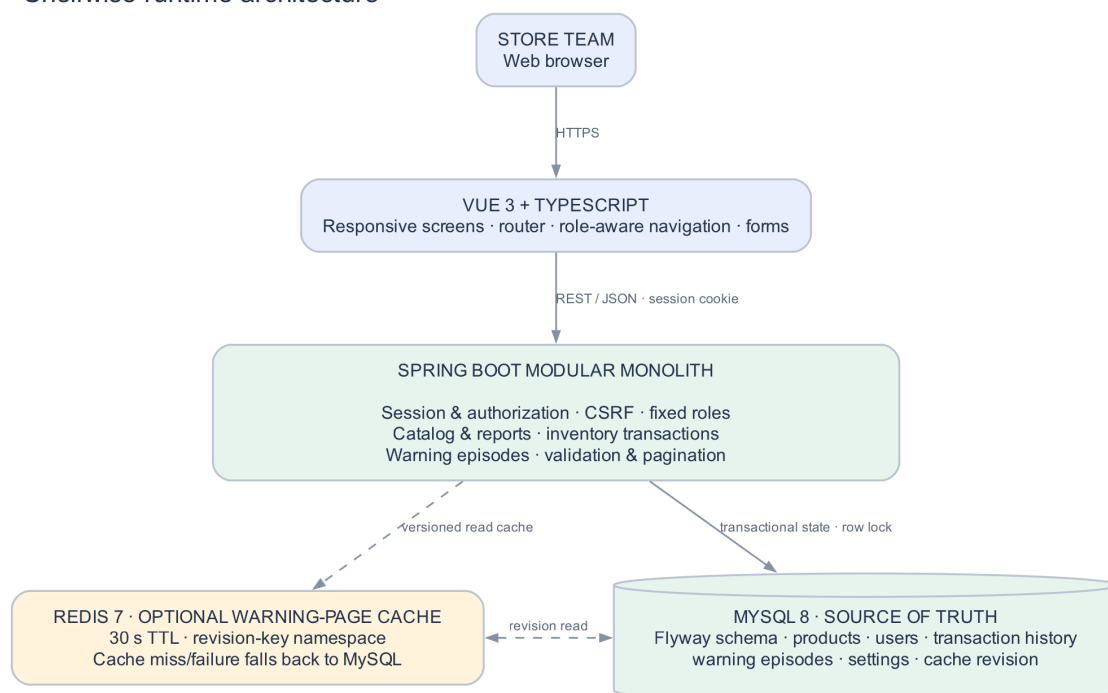


图 1: Shelfwise 系统运行架构。MySQL 保存持久状态，Redis 只负责可选查询缓存。

选择模块化单体而不是拆分多个服务，原因是项目范围只有一个门店，库存和预警又共享同一个事务边界。拆分服务会引入消息延迟、重复消费和跨服务一致性问题，但当前没有证据表明系统需要这种复杂度。

3.2 数据库模型

核心实体包括用户、商品、分类、供应商、库存事务、预警、设置和缓存版本。商品表保存当前数量，库存事务表保存变化解释，二者在同一个事务中更新。事务表使用“操作人 + 幂等键”唯一约束，商品 SKU 和条码使用唯一约束。商品有历史记录后只能归档，不能直接删除；归档不会伪造出库流水。

Core relational data model

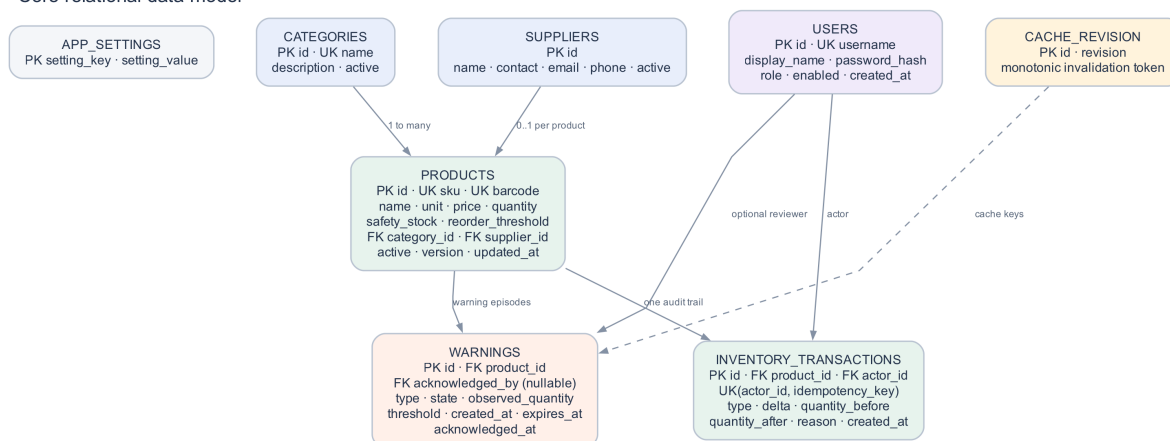


图 2: 核心实体关系。库存流水和预警均引用商品，操作人负责提供归因和确认信息。

3.3 库存事务算法

一次入库、出库或盘点请求依次经过：身份和角色校验、输入校验、产品行锁、幂等键检查、库存边界检查、库存与流水写入、预警更新以及缓存版本递增。任何一步失败，整个数据库事务回滚。

行锁解决并发出库问题。例如库存只有 1 件时，两个并发出库请求不能都读取到旧值 1 并同时成功。第一个请求提交后，第二个请求读取到最新数量 0 并被拒绝。幂等键解决网络重试问题：相同操作重试时返回原流水；相同键但内容不同则返回冲突。

Stock movement: transaction and warning workflow

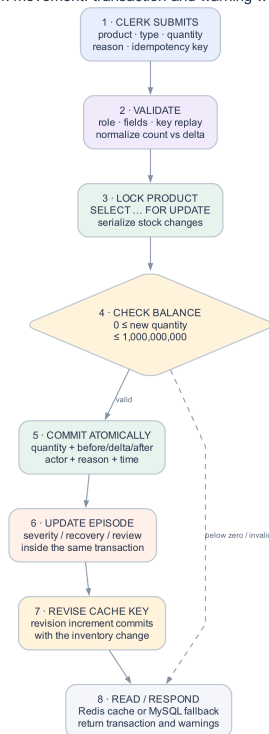


图 3: 库存事务和预警更新流程。库存、流水、预警版本在同一事务边界内处理。

3.4 预警生命周期与缓存

预警生命周期和人工确认分开存储。反复查询同一缺货事件不会产生无限重复预警；严重程度变化会关闭旧事件并开启新事件；库存恢复会产生短期 RESTOCKED 事件。Redis 键包含数据库缓存版本、预警类型、状态、页码和页大小。库存、阈值或确认发生提交时，数据库版本递增，后续读取自然进入新命名空间。旧缓存由 TTL 回收，而不是清空整个 Redis。

4 系统实现

4.1 认证和权限

系统使用服务端 Session、CSRF Token 和 BCrypt 密码哈希。浏览器先获取 CSRF Token，再提交登录或其他写操作。用户状态和角色在请求期间从 MySQL 重新确认，因此暂停账号或修改角色后不依赖用户主动退出登录才能生效。演示账号和密码来自被 Git 忽略的本地环境文件。

4.2 库存和预警界面

库存页面支持按名称、SKU、条码、分类和库存状态搜索，并提供分页和稳定排序。库存事务表单支持收货、出库、调整和盘点，并要求填写原因。预警页面同时展示商品、当前数量、阈值、首次发现时间和确认状态。报表页面把当前库存数量、当前价格和交易日期聚合成可读指

标，但明确说明这些数据不是会计估值或销售预测。

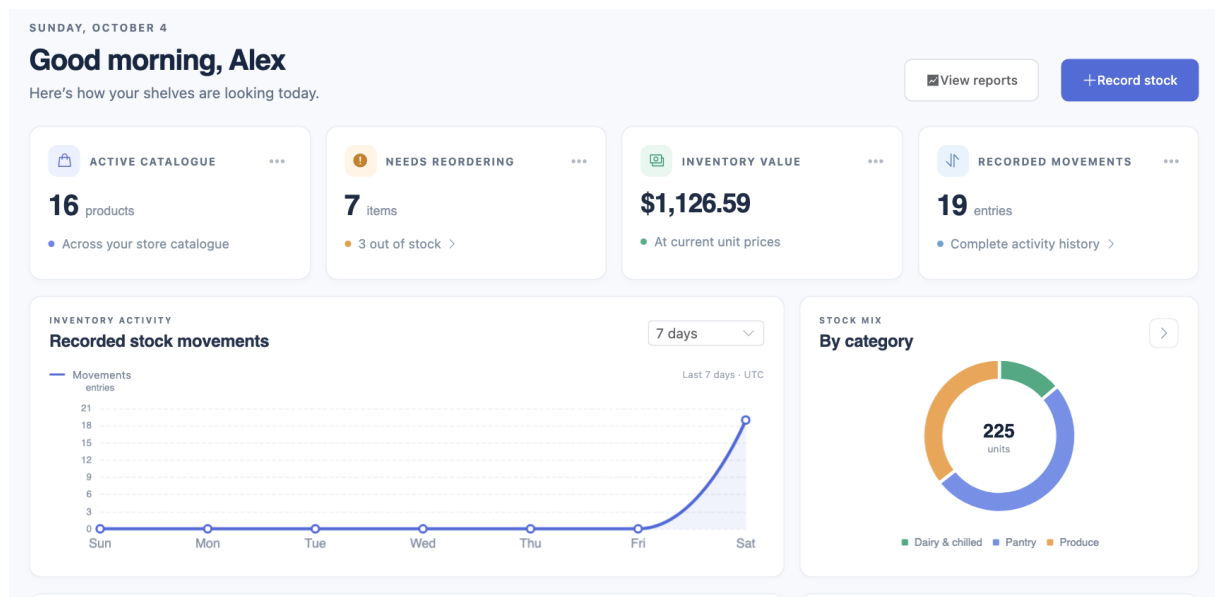


图 4: 系统首页仪表盘，展示当前目录、库存预警、库存估值估计和记录的事务数量。

The **Record stock activity** form is shown over a table of stock activity. The form fields are:

- Product:** Search your catalogue (dropdown)
- Activity type:** Receive stock (dropdown)
- Quantity:** 1 (input field)
- Reason for this activity:** e.g. Morning delivery, damaged items, weekly count (text area, 0/300 characters)

The form includes **Cancel** and **Record activity** buttons. The background table lists stock activity with columns for Product, Quantity, Reason, and Date & Time.

Product	Quantity	Reason	Date & Time
Apples, Gala	+38	Opening demonstration delivery	Oct 4, 2:26 AM
Olive oil 500ml	+1	Opening demonstration delivery	Oct 4, 2:12 AM
Tomato soup 400g	+1	Opening demonstration delivery	Oct 4, 2:12 AM
Rice 2kg	+1	Opening demonstration delivery	Oct 4, 2:12 AM
Cheddar cheese	+1	Opening demonstration delivery	Oct 4, 2:12 AM
Carrots 1kg	+1	Opening demonstration delivery	Oct 4, 2:12 AM
Apples, Gala	+38	Opening demonstration delivery	Oct 4, 2:12 AM
Tomatoes, vine	+1	Role test	Oct 4, 1:58 AM
Tomatoes, vine	-1	Acceptance smoke	Oct 4, 1:58 AM
Tomatoes, vine	-1	Acceptance smoke	Oct 4, 1:57 AM
Tomatoes, vine	-1	Acceptance smoke	Oct 4, 1:28 AM
Tomatoes, vine	-1	Acceptance smoke	Oct 4, 1:27 AM

图 5: 库存事务表单。表单提交的是操作意图，数量前后值由服务器在事务中计算。

4.3 实现中的修正

编译通过并不等于系统行为正确。开发过程中发现并修正了 Vite 依赖缓存、ECharts 渲染器注册、库存流水懒加载、Java 时间序列化和环境变量映射等问题。这些问题说明系统必须同时经过源代码检查、数据库接口调用和浏览器运行检查。

5 测试与实验

5.1 测试结果

后端使用真实 MySQL 和 Redis 容器运行 27 项集成测试，覆盖权限、CSRF、阈值边界、重复提交、并发出库、归档、预警确认、管理员保护和输入校验。前端有 3 项库存状态单元测试。浏览器脚本完成 14 项检查，包括登录、库存搜索、库存登记、预警确认、角色跳转和移动端布局，未发现 JavaScript 运行时错误。

表 3: 主要验证结果

验证层	结果	说明
后端集成测试	27 项通过	使用真实 MySQL/Redis，覆盖事务、权限、并发和预警
前端单元测试	3 项通过	覆盖零库存、等于阈值和健康库存
浏览器流程	14 项通过	页面渲染、库存登记、预警确认和移动端检查
Redis 故障恢复	通过	Redis 停止期间仍返回同一数据库结果
应用重启持久性	通过	重启前后库存与流水指纹一致

5.2 缓存实验

缓存实验使用同一个认证用户、同一个接口、同一个页大小和同一个演示数据集。缓存路径和直接查询路径各采集 600 个请求样本。缓存路径的中位延迟约为 5.33 ms，直接查询约为 6.56 ms；缓存和非缓存响应经过 JSON 等价性比较。该结果只能说明在本地小数据集上缓存减少了一部分查询工作，不能推导出生产环境 SLA，也不能证明所有小型超市都必须使用 Redis。

5.3 故障与持久性

停止 Redis 后，预警接口仍通过 MySQL 返回相同的商品、数量和确认信息；Redis 恢复后查询结果保持一致。应用重启后，商品数量和库存流水的有序指纹不变。这两个实验支持“Redis 不是写入正确性的前提”和“正常应用重启不丢失库存历史”这两个有限结论，但不等于已经完成磁盘损坏恢复或生产灾备验证。

6 讨论、局限与结论

项目证明了一个单店库存演示系统可以把库存余额、流水历史、预警生命周期和人工确认组织在同一套可复现实现中。行锁、幂等键和数据库约束共同保护库存不变量；预警确认不再冒充补货；缓存版本让查询加速不改变 MySQL 的权威地位。

项目没有证明降低了真实缺货损失，也没有做真实门店用户研究。库存使用整数单位，没有批次和有效期，阈值由人工配置，没有多店部署、自动采购、财务核算和需求预测。下一步应优先进行门店任务观察、安全和备份恢复评估、分数单位和批次支持、POS 接入以及更大规模的并发实验，而不是继续堆叠新的技术组件。

因此，论文的结论不是“系统已经适用于所有超市”，而是：在明确的单店范围和本地验证环境内，Shelfwise 实现了可追溯库存事务、可解释预警和可回退缓存路径，并且这些结论可以由源代码、数据库测试、浏览器证据和实验数据逐项复核。

阅读对照建议

阅读英文论文时，可以依次检查四条线：第一，绪论中的研究问题是否在需求和测试中有对应物；第二，设计章节中的库存不变量是否在实现和集成测试中落地；第三，Redis 性能结果是否同时报告了缓存一致性和故障回退；第四，结论是否只复述已经验证的内容，并主动列出真实部署仍然缺少的证据。若这四条线闭合，论文的整体论证就成立。